Дисциплина «API-технологии»

Методические указания к практической работе № 2

Проектирование REST API для сервиса прогнозирования спроса или урожайности

1. Для чего нужна эта работа

На предыдущем занятии вы описали бизнес-систему: кто пользуется её данными, какие ресурсы нужны и как участники связаны друг с другом. Теперь предстоит сделать следующий шаг — превратить словесные связи в проект API. Клиент должен понимать, куда обратиться, какие данные передать и что означают полученные ответ или ошибка.

Эта работа не требует писать Python-код, обучать модель или запускать сервер. Вы создаёте проект контракта REST API для одного понятного сценария. На следующей практике вы научитесь отправлять настоящие HTTP-запросы и проверять ответы, а позднее реализуете сервис на FastAPI.

2. Что вы должны получить в результате

За два академических часа (90 минут) подготовьте краткое описание выбранного сценария и границ сервиса, таблицу из 3–4 операций, примеры входных и выходных данных для двух основных операций, описание как минимум двух типовых ошибок и одного правила совместимости. Используйте контекстную схему и таблицу интерфейсов из практической работы № 1: каждый предложенный маршрут должен быть нужен конкретному потребителю.

На занятии сдаётся проект контракта в заполненном бланке. В домашнем задании № 2 вы перенесёте согласованный вариант в выданный файл-шаблон OpenAPI. Полноценную спецификацию с нуля за одно занятие составлять не требуется. В этой работе мы рассматриваем только сервис прогнозирования спроса либо урожайности; кредитный скоринг и интеграция ERP/CRM/BI могут оставаться частью общей контекстной схемы, но не заменяют тему практики.

3. Как выполнять работу

Шаг 1. Выберите сценарий и определите границы сервиса.

Выберите прогноз спроса для розничной сети или прогноз урожайности для агропредприятия. В двух-трёх предложениях укажите, кто принимает решение на основе прогноза и какая система предоставляет исходные данные. Отдельно отметьте, что входит в ваш сервис, а что является чужой системой: её API не нужно автоматически включать в ваш контракт.

> *Пример решения. Сеть агромагазинов планирует закупки семян. Менеджер через внутреннее приложение запрашивает прогноз продаж конкретного товара на будущий*

месяц. Сервис прогнозов создаёт и хранит результаты; историю продаж он получает из учётной системы, которая остаётся отдельным владельцем данных. На этой практике расчёт прогноза условный — качество ML-модели не оценивается.

Шаг 2. Выделите ресурсы и составьте операции.

Возьмите необходимые связи с вашей схемы из ПР № 1. Назовите ресурс существительным: «прогноз», а не «рассчитать прогноз». Для небольшого учебного API достаточно 3–4 операций. У каждой должны быть HTTP-метод, путь, потребитель и понятное назначение. GET получает данные, POST передаёт данные для создания нового результата; не нужно добавлять все методы CRUD только ради количества.

Пример решения. Менеджер (через приложение) пользуется одним сервисом прогнозов. Операции этого сервиса: POST /forecasts — создать прогноз по товару, магазину и периоду; GET /forecasts/{forecast_id} — прочитать прогноз по идентификатору; GET /forecasts?product_id=12 — получить список прогнозов для товара. Историю продаж сервис получает из внешней учётной системы: соответствующий маршрут, если он есть, принадлежит ей, а не вашему API.

Шаг 3. Опишите входные данные и успешные ответы.

Для двух основных операций из своей таблицы запишите понятные примеры. Если параметры передаются в пути или после знака вопроса, укажите их; если передаётся тело запроса — приведите небольшой JSON. Укажите код успешного ответа и состав результата. Названия полей в примерах должны оставаться одинаковыми во всех операциях.

Пример решения. Для POST /forecasts тело запроса: {"product_id": 12, "store_id": 3, "period": "2026-10"}. Успешный ответ — код 201 и JSON: {"forecast_id": 101, "product_id": 12, "store_id": 3, "period": "2026-10", "predicted_units": 85}. Для GET /forecasts/101 тело запроса не требуется; успешный ответ — код 200 и те же поля прогноза. Числа и идентификаторы здесь вымышлены.

Шаг 4. Предусмотрите ошибки и ограничения.

Подумайте, как клиент узнает, что запрос нельзя выполнить. Достаточно двух типовых ситуаций. Назовите причину, код ответа и пример понятного сообщения. Затем зафиксируйте необходимые ограничения для полей, чтобы клиент не гадал об их смысле и формате.

Пример решения. Если в POST /forecasts отсутствует обязательное поле period, в учебном контракте возвращается код 400 и {"error": "period is required"}. Если GET /forecasts/999 запрашивает несуществующий прогноз — код 404 и {"error": "forecast not found"}. Поле period задаётся как год-месяц в формате YYYY-MM, predicted_units означает целое число единиц товара. Это проектируемое правило нашего примера, а не утверждение о поведении готового сервера.

Шаг 5. Проверьте совместимость и связь с ПР № 1.

Сравните вашу таблицу операций с контекстной схемой: кто вызывает каждую операцию и для чего? Не включили ли вы маршрут внешней системы в собственный API? Представьте, что другое приложение уже читает поле predicted_units. Подумайте, что произойдёт, если это поле внезапно переименовать.

Пример решения. POST /forecasts и GET /forecasts/{forecast_id} вызывает приложение менеджера — соответствующая стрелка есть в схеме ПР № 1. История продаж поступает из отдельной учётной системы, поэтому её интерфейс не описывается как маршрут сервиса прогнозов. Правило совместимости: существующие поля ответа, которыми пользуются клиенты, нельзя переименовывать без изменения версии контракта или согласования с клиентами.

Шаг 6. Подготовьте итоговый контракт к сдаче.

Проверьте, что в заполненном бланке есть краткий сценарий, 3–4 согласованные операции, входные и выходные данные двух операций, успешные и ошибочные ответы, ограничения и правило совместимости. В конце занятия кратко объясните свои решения. После согласования перенесите те же сведения в шаблон OpenAPI как домашнее задание № 2: методов и путей в спецификации не должно быть больше «просто для красоты».

Пример решения. В строке итоговой таблицы: «Менеджер через приложение; ресурс — прогноз; POST /forecasts; передаёт product_id, store_id, period; получает forecast_id и predicted_units; успех — 201, неверные входные данные — 400». Эту строку можно использовать как опору при заполнении раздела paths в шаблоне OpenAPI. Тестирование запроса в Postman здесь не требуется: оно относится к следующему занятию.

4. Защита работы

Покажите заполненный бланк и объясните, какой бизнес-вопрос решает API, почему выбран именно ресурс «прогноз» (или иной ресурс вашего варианта), как клиент узнаёт об ошибке и какая связь из ПР № 1 подтверждает необходимость каждого маршрута. Защищается проект, а не работоспособность кода. Если преподаватель попросит изменить контракт, внесите это изменение до оформления домашнего задания в OpenAPI.

5. На что обратить внимание

Не путайте ресурс с действием и не смешивайте интерфейсы разных систем. Один и тот же идентификатор прогноза должен обозначаться одинаково в пути, примере ответа и описании ошибки. Поля JSON должны иметь понятный смысл; у числа прогнозируемых единиц должны быть указаны единицы измерения. Ошибочный ответ — часть контракта, а не необязательное дополнение.

Главный признак качественно выполненной работы прост: другой человек, не видя вашего будущего кода, сможет по бланку понять, какой запрос отправить, что получить в нормальной ситуации и что делать при ошибке. Сохраните этот проект: он понадобится при разработке сервиса на следующих практических занятиях.